

ADVANCED WEB PROGRAMMING
ELOQUENT ORM



KHEN MUHAMMAD CAHYO

244107023002

TI-21

STATE POLYTECHNIC OF MALANG
INFORMATION TECHNOLOGY DEPARTMENT

PRACTICUM 1 (FILLABLE & GUARD)

- Practicum 1-3

```
protected $table = 'users';
protected $primaryKey = 'user_id';
protected $fillable = [
    'user_id',
    'level_id',
    'username',
    'name',
    'password',
];

public function store() {
    $operation = User::create([
        'level_id' => fake()->numberBetween(1, 3),
        'name' => fake()->name(),
        'username' => fake()->unique()->userName,
        'password' => bcrypt('123456'),
    ]);

    if (!$operation) {
        return 'Failed to insert data!';
    }

    return 'Successfully insert data!';
}
```

The \$fillable property in the model specifies which attributes can be mass-assigned. In the store() method, a new User record is created using Laravel's User::create() method (ORM), with level_id, name, username, and password populated using Faker for random values. The password is encrypted using bcrypt().

- Practicum 4-6

```
protected $fillable = [
    'user_id',
    'level_id',
    'username',
    'name',
];
```

If we remove password from the \$fillable array, Laravel will prevent mass assignment of the password field when using User::create() ORM. This means the store() method will fail to insert the password, causing an error or ignoring the field.

PRACTICUM 2.1 (RETRIEVING SINGLE MODELS)

- Practicum 1-3

```
public function show(string $id) {  
    $user = User::find($id);  
  
    return view('pages.user.details', compact('user'));  
}
```

User Details Data

ID	Username	Name	Level ID
1	admin	Administrator	1

The `show()` method retrieves a `User` record by its `$id` using `User::find($id)` and passes it to the `pages.user.details` view using the `compact('user')` function. In the Blade template, the `<tbody>` section dynamically displays the `user_id`, `username`, `name`, and `level_id` of the retrieved user. However, if `User::find($id)` returns null (e.g., if the user does not exist), this code may cause an error when trying to access properties on a null object.

- Practicum 4-5

```
public function show(string $id) {  
    $user = User::where('level_id', $id)->first();  
  
    return view('pages.user.details', compact('user'));  
}
```

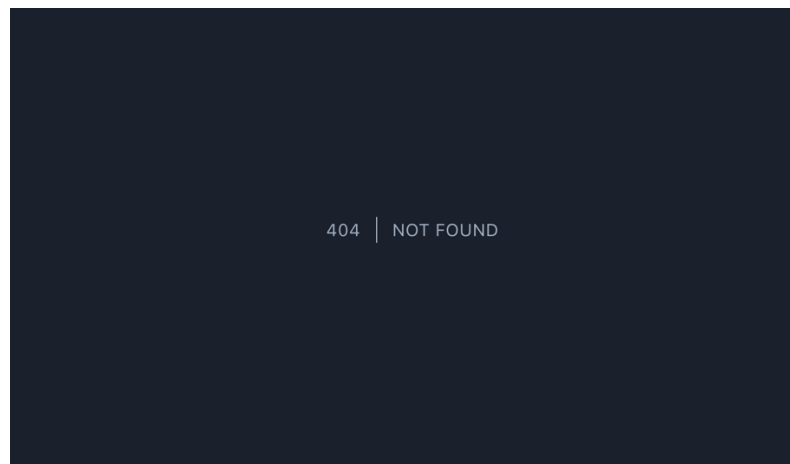
The `show()` method retrieves the first `User` record where the `level_id` matches the provided `$id` using `User::where('level_id', $id)->first()`. The retrieved user is then passed to the `pages.user.details` view using `compact('user')`. However, this approach assumes `$id` represents a `level_id` rather than a unique `user_id`, which might lead to unintended results if multiple users share the same `level_id`. Additionally, if no matching user is found, `$user` will be null.

- **Practicum 6-7**

```
public function show(string $id) {  
    $user = User::firstWhere('level_id', $id);  
  
    return view('pages.user.details', compact('user'));  
}
```

The `show()` method retrieves the first `User` record where `level_id` matches the provided `$id` using `User::firstWhere('level_id', $id)`, which is a shorthand for `where()->first()`. The retrieved user is then passed to the `pages.user.details` view using `compact('user')`. However, this approach assumes `$id` corresponds to a `level_id` rather than a unique `user_id`, which may lead to unexpected results if multiple users share the same `level_id`. Additionally, if no matching user is found, `$user` will be null

- **Practicum 8-11**



```
public function show(string $id) {  
    $user = User::findOrFail($id, ['username', 'name'], function() {  
        abort(404);  
    });  
  
    return view('pages.user.details', compact('user'));  
}
```

The `show()` method attempts to retrieve a `User` record with an `id` of `$id` while selecting only the `username` and `name` columns using `User::findOrFail(1, ['username', 'name'], function() { abort(404);`), ensuring that if no record is found, a 404 error is automatically returned.

PRACTICUM 2.2 (NOT FOUND EXCEPTIONS)

- **Practicum 1-2**

```
public function show(string $id) {  
    $user = User::findOrFail($id);  
  
    return view('pages.user.details', compact('user'));  
}
```

The show() method retrieves a User record by its \$id using User::findOrFail(\$id). If a user with the given id exists, it is passed to the pages.user.details view using compact('user'). However, if no user is found, Laravel automatically throws a 404 Not Found error instead of returning null, preventing potential errors in the view when accessing user properties. This approach ensures that only valid users are displayed and provides a proper error response if the requested user does not exist.

- **Practicum 3-4**

```
public function show(string $id) {  
    $user = User::where('username', 'abc')->firstOrFail();  
  
    return view('pages.user.details', compact('user'));  
}
```

The show() method retrieves the first User record where the username is 'abc' using firstOrFail(), which throws a 404 Not Found error if no match is found. This differs from findOrFail(\$id), which searches by id, whereas this method always looks for the hardcoded username 'abc', making the \$id parameter ineffective. To dynamically fetch a user by username, the query should use where('username', \$id).

PRACTICUM 2.3 (RETRIEVING AGGREGATES)

- Practicum 1-2

```
1 // app/Http/Controllers/UserController.php:12
```

```
public function index() {  
    $users = User::where('level_id', 2)->count();  
    dd($users);  
    return view('pages.user.index', compact('users'));  
}
```

The `index()` method retrieves the count of User records where `level_id` is 2 using `User::where('level_id', 2)->count()`. The result, which is an integer representing the number of matching users, is stored in `$users`. The `dd($users);` (dump and die) statement then stops execution and outputs the count for debugging purposes. Because of this, the `return view('pages.user.index', compact('users'));` statement will never execute. To properly display the count in the view, the `dd()` statement should be removed.

- Practicum 3-4

Users Data

Total Users: 1

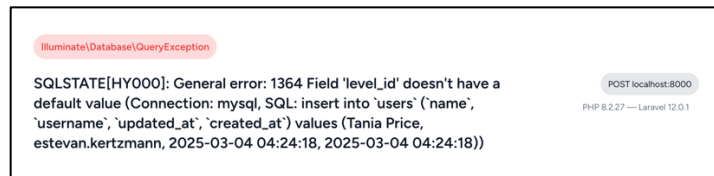
```
<p>Total Users: {{ $totalUsers }}</p>
```

```
public function index() {  
    $users = User::all();  
    $totalUsers = User::where('level_id', 2)->count();  
    return view('pages.user.index', compact('users', 'totalUsers'));  
}
```

The `index()` method retrieves all users and counts those with `level_id` of 2, passing both to the view. The Blade template then displays the total count using `<p>Total Users: {{ $totalUsers }}</p>`.

PRACTICUM 2.4 (RETRIEIVING OR CREATING MODELS)

- Practicum 1-3



```
public function store() {
    $operation = User::firstOrCreate([
        'name' => fake()->name(),
        'username' => fake()->unique()->userName,
    ]);

    return redirect()->route('user.index');
}
```

The store() method will give us errors because level_id and password don't have default value in database and it's not nullable columns. So we must create value for level_id and password also.

- Practicum 4-5

```
public function store() {
    $operation = User::firstOrCreate([
        'level_id' => fake()->numberBetween(1, 3),
        'name' => fake()->name(),
        'username' => fake()->unique()->userName,
        'password' => bcrypt('123456'),
    ]);

    return redirect()->route('user.index');
}
```

Users Data

Total Users: 1

[Add New User](#)

ID	Username	Name	Action
1	admin	Administrator	Delete Update
2	manager	Manager	Delete Update
3	staff	Staff/Kasir	Delete Update
4	aracely.frami	Fredy Hammes MD	Delete Update

The store() method attempts to create a new User record using firstOrCreate(), which checks if a user with the given attributes already exists; if not, it creates a new one. It generates random level_id, name, and username using Faker, while the password is encrypted with bcrypt('123456').

- **Practicum 6-7**

```
public function store() {  
    $operation = User::firstOrCreate([  
        'name' => fake()->name(),  
        'username' => fake()->unique()->userName,  
    ]);  
  
    return redirect()->route('user.index');  
}
```

The store() method attempts to find a User with the given name and username using firstOrCreate(), which returns an existing user if found or creates a new instance without saving it to the database. However, since firstOrCreate() does not automatically persist the record, the new user will not be saved unless save() is explicitly called.

- **Practicum 8-9**

```
public function store() {  
    $operation = User::firstOrCreate([  
        'level_id' => fake()->numberBetween(1, 3),  
        'name' => fake()->name(),  
        'username' => fake()->unique()->userName,  
        'password' => bcrypt('123456'),  
    ]);  
  
    return redirect()->route('user.index');  
}
```

The store() method attempts to find a User with the given level_id, name, username, and password using firstOrCreate(). If a matching record exists, it returns that instance; otherwise, it creates a new one but does **not** save it to the database unless save() is explicitly called.

- **Practicum 10-11**

```
public function store() {  
    $operation = User::firstOrCreate([  
        'level_id' => fake()->numberBetween(1, 3),  
        'name' => fake()->name(),  
        'username' => fake()->unique()->userName,  
        'password' => bcrypt('123456'),  
    ]);  
    $operation->save();  
  
    return redirect()->route('user.index');  
}
```

The store() method finds or creates a User without saving it immediately, then calls save() to persist it in the database before redirecting to user.index.

PRACTICUM 2.5 (ATTRIBUTE CHANGES)

- Practicum 1-2

```
false // app/Http/Controllers/UserController.php:41
```

```
public function store() {  
    $operation = User::create([  
        'level_id' => fake()->numberBetween(1, 3),  
        'name' => fake()->name(),  
        'username' => fake()->unique()->userName(),  
        'password' => bcrypt('123456'),  
    ]);  
  
    $operation->username = 'manage45';  
  
    $operation->isDirty();  
    $operation->isDirty('username');  
    $operation->isDirty('name');  
    $operation->isDirty(['name', 'username']);  
  
    $operation->isClean();  
    $operation->isClean('username');  
    $operation->isClean('name');  
    $operation->isClean(['name', 'username']);  
  
    $operation->save();  
  
    $operation->isDirty();  
    $operation->isClean();  
    dd($operation->isDirty());  
  
    return redirect()->route('user.index');  
}
```

The store() method creates a new User record and assigns it to \$operation. The username is then updated to 'manage45', making the model "dirty" (modified but not saved). Several isDirty() and isClean() checks are performed to determine if specific attributes (username, name) or the entire model have unsaved changes. After calling save(), the model becomes "clean" (no pending changes). Another set of isDirty() and isClean() checks is performed before dd(\$operation->isDirty()) dumps the final state, preventing further execution.

- Practicum 3-4

```
true // app/Http/Controllers/UserController.php:33
```

```
public function store() {  
    $operation = User::create([  
        'level_id' => fake()->numberBetween(1, 3),  
        'name' => fake()->name(),  
        'username' => fake()->unique()->userName(),  
        'password' => bcrypt('123456'),  
    ]);  
  
    $operation->username = 'manage400';  
  
    $operation->save();  
  
    $operation->wasChanged();  
    $operation->wasChanged('username');  
    $operation->wasChanged(['username', 'level_id']);  
    $operation->wasChanged('name');  
    dd($operation->wasChanged(['username', 'level_id']));  
  
    return redirect()->route('user.index');  
}
```

The store() method creates a new User record and assigns it to \$operation. The username is then updated to 'manage400', and save() is called to persist the change. Several wasChanged() checks are performed to determine if specific attributes (username, level_id, name) were modified after saving.

PRACTICUM 2.6 (CRUD)

- Practicum 1-3

```
public function index() {  
    $users = User::all();  
  
    return view('pages.user.index', compact('users'));  
}
```

```
Route::prefix('/user')->group(function () {  
    // Page Routes  
    Route::get('/', [UserController::class, 'index']->name('user.index'));
```

```
<body>  
    <h1>Users Data</h1>  
    <a href="{{ route('user.store-page') }}">Create New User</a>  
    <table border="1" cellpadding="2" cellspacing="0">  
        <thead>  
            <tr>  
                <th>ID</th>  
                <th>Username</th>  
                <th>Name</th>  
                <th>Action</th>  
            </tr>  
        </thead>  
        <tbody>  
            @foreach ($users as $user)  
                <tr>  
                    <td>{{ $user->user_id }}</td>  
                    <td>{{ $user->username }}</td>  
                    <td>{{ $user->name }}</td>  
                    <td>  
                        <form action="{{ route('user.delete', ['id' => $user->user_id]) }}"  
                            method="POST">  
                            @method('DELETE')  
                            @csrf  
                            <button type="submit">Delete</button>  
                        </form>  
                        <a href="{{ route('user.update-page', ['id' => $user->user_id]) }}">Update</a>  
                    </td>  
                </tr>  
            @endforeach  
        </tbody>  
    </table>  
</body>
```

Users Data

[Create New User](#)

ID	Username	Name	Action
3	staff	Staff/Kasir	Delete Update
4	aracely.frami	Fredy Hammes MD	Delete Update
5	adelbert.ullrich	Trever Jones II	Delete Update
6	habshire	Mr. Karley Pfeffer	Delete Update
7	manage45	Prof. Bryon Johnson Jr.	Delete Update
8	chet53	Dr. Demarcus Bartoletti	Delete Update
9	manage400	Mr. Bradley Homenick	Delete Update
10	nleannon	Dr. Lesley Auer	Delete Update
12	ABC	ABCDE 1	Delete Update

The index() method retrieves all users and passes them to the pages.user.index view. The view displays a table listing user data with options to update or delete each user. A link is provided to create a new user. The DELETE method is used for deletions, protected by CSRF. The route group prefixes /user and includes the index route for displaying users.

- **Practicum 4-10**

```
CodeIgniter Refactor | Explain | Generate Function Comment | X
public function storePage() {
    return view('pages.user.store');
}
```

```
Route::post('/', [UserController::class, 'store'])->name('user.store');
Route::get('/store', [UserController::class, 'storePage'])->name('user.store-page');
```

```
CodeIgniter Refactor | Explain | Generate Function Comment | X
public function store(Request $request) {
    $operation = User::create([
        'level_id' => $request->level_id,
        'name' => $request->name,
        'username' => $request->username,
        'password' => bcrypt($request->password),
    ]);

    if (!$operation) {
        return 'Failed to insert data!';
    }

    return redirect()->route('user.index');
}

<body>
<h1>Create a New User</h1>
<form action="{{ route('user.store') }}" method="POST">
    @csrf
    <label for="username">Username</label>
    <input type="text" name="username" id="username">
    <br><br>
    <label for="name">Name</label>
    <input type="text" name="name" id="name">
    <br><br>
    <label for="password">Password</label>
    <input type="password" name="password" id="password">
    <br><br>
    <label for="level_id">Level ID</label>
    <input type="text" name="level_id" id="level_id">
    <br><br>
    <button type="submit">Submit</button>
</form>
</body>
```

Create a New User

Username

Name

Password

Level ID

13	USER BARU	BARU
----	-----------	------

[Update](#)

The storePage() method returns the user creation form, which submits data via POST to the store() method. This method creates a new user with the provided level_id, name, username, and hashed password. If creation fails, an error message is returned; otherwise, it redirects to the user list. The form includes CSRF protection and input fields for user details. The routes define paths for displaying and processing the form.

- Practicum 11-17

```
Codeium: Refactor | Explain | Generate Function Comment | X
public function updatePage(string $id) {
    $user = User::findOrFail($id);

    return view('pages.user.update', compact('user'));
}
```

```
Codeium: Refactor | Explain | Generate Function Comment | X
public function update(Request $request, string $id) {
    $operation = User::where('user_id', $id)->update([
        'level_id' => $request->level_id,
        'name' => $request->name,
        'username' => $request->username,
    ]);

    if (!$operation) {
        return 'Failed to update data!';
    }

    return redirect()->route('user.index');
}
```

```
laravel: update: user /id/
<form action="{{ route('user.update', ['id' => $user->user_id]) }}" method="POST">
    @csrf
    @method('PATCH')
    <label for="username">Username</label>
    <input type="text" name="username" id="username" value="{{ $user->username }}">
    <br><br>
    <label for="name">Name</label>
    <input type="text" name="name" id="name" value="{{ $user->name }}">
    <br><br>
    <label for="level_id">Level ID</label>
    <input type="text" name="level_id" id="level_id" value="{{ $user->level_id }}">
    <br><br>
    <button type="submit">Submit</button>
</form>
```

```
laravel: update: user /id/
username: b9cfμ(, \{q}, ' [ηζεζουφοορρε::cf922', nbq9f6, ])->u9w6(, n26L·nbq9f6, )!
username: δεε(, \{q}\nbq9f6, ' [ηζεζουφοορρε::cf922', nbq9f6b9δ6, ])->u9w6(, n26L·nbq9f6-b9δ6, )!
```

Update User

Username

Name

Level ID

6	habshire update	Mr. Karley Update	Delete Update
---	-----------------	-------------------	---

The `updatePage()` method retrieves a user by ID and displays the update form pre-filled with their data. The form submits a PATCH request to the `update()` method, which updates the user's `level_id`, `name`, and `username`. If the update fails, an error message is returned; otherwise, it redirects to the user list. Routes define the paths for displaying and processing updates.

- **Practicum 18-20**

```
public function delete(string $id) {  
    $operation = User::destroy('user_id', $id);  
  
    if (!$operation) {  
        return 'Failed to delete data!';  
    }  
  
    return 'Successufully delete data!';  
}
```

```
Route::delete('/{id}', [UserController::class, 'delete'])->name('user.delete');
```

Successufully delete data!

The delete() method attempts to delete a user by their user_id using User::destroy('user_id', \$id), but this is incorrect because destroy() expects an ID directly, not a column name. If the deletion fails, it returns a failure message; otherwise, it confirms success. The corresponding DELETE route maps to this method, allowing users to be deleted via an HTTP DELETE request.

PRACTICUM 2.7 (RELATIONSHIP)

- Practicum 1-3

```
Illuminate\Database\Eloquent\Collection {#291 ▼ // app/Http/Controllers/UserController.php:12
  #items: array:10 [▼]
    0 => App\Models\User {#296 ▶}
    1 => App\Models\User {#297 ▶}
    2 => App\Models\User {#298 ▶}
    3 => App\Models\User {#299 ▶}
    4 => App\Models\User {#300 ▶}
    5 => App\Models\User {#301 ▶}
    6 => App\Models\User {#302 ▶}
    7 => App\Models\User {#303 ▶}
    8 => App\Models\User {#304 ▶}
    9 => App\Models\User {#305 ▶}
  }
  #escapeWhenCastingToString: false
}
```

```
public function index() {
    $users = User::with('level')->get();
    dd($users);

    return view('pages.user.index', compact('users'));
}
```

The `index()` method retrieves all users along with their associated level data using Eloquent's `with()` to eager load the relationship, then dumps the result with `dd($users)`. The `level()` method defines a `belongsTo` relationship, linking each user to a `Level` model based on `level_id`.

- Practicum 4-6

Users Data

[Create New User](#)

ID	Username	Name	Level Code	Level Name	Action
3	staff	Staff/Kasir	STF	Staff/Kasir	Delete Update
4	aracely.frami	Fredy Hammes MD	ADM	Administrator	Delete Update
5	adelbert.ullrich	Trever Jones II	ADM	Administrator	Delete Update
6	habshire update	Mr. Karley Update	MNG	Manager	Delete Update
7	manage45	Prof. Bryon Johnson Jr.	STF	Staff/Kasir	Delete Update
8	chet53	Dr. Demarcus Bartoletti	STF	Staff/Kasir	Delete Update
9	manage400	Mr. Bradley Homenick	ADM	Administrator	Delete Update
10	nleannon	Dr. Lesley Auer	MNG	Manager	Delete Update
12	ABC	ABCDE 1	ADM	Administrator	Delete Update
13	USER BARU	BARU	ADM	Administrator	Delete Update

```
<table border="1" cellpadding="2" cellspacing="0">
<thead>
<tr>
<th>ID</th>
<th>Username</th>
<th>Name</th>
<th>Level Code</th>
<th>Level Name</th>
<th>Action</th>
</tr>
</thead>
<tbody>
<foreach $users as $user>
<tr>
<td>{{ $user->user_id }}</td>
<td>{{ $user->username }}</td>
<td>{{ $user->name }}</td>
<td>{{ $user->level->level_code }}</td>
<td>{{ $user->level->level_name }}</td>
<td>
<form action="{{ route('user.delete', ['id' => $user->user_id]) }}"
method="POST">
@method('DELETE')
@csrf
<button type="submit">Delete</button>
</form>
<a href="{{ route('user.update-page', ['id' => $user->user_id]) }}">Update</a>
</td>
</tr>
</foreach>
</tbody>
</table>
```

```
public function index() {
    $users = User::with('level')->get();

    return view('pages.user.index', compact('users'));
}
```

The index() method retrieves all users along with their associated level data using Eloquent's with('level') for eager loading, then passes the data to the pages.user.index view. In the view, a table displays user details, including their ID, username, name, and related level_code and level_name from the level relationship. Each row includes options to delete or update the user using a form with a DELETE request and a link to the update page.



KhenCahyo13 AWP Week 4 - ORM